Computational Physics Lectures

Week 8: Regression

Bjørk Hammer
Department of Physics and Astronomy
Aarhus University
Denmark

Contents

8 Regression **3**

8.1 Ordinary Least Squares (OLS) 3
8.2 Maximization of Log-Likelihood 4
8.3 Linear regression example 5
8.4 Ridge Regression 5
8.5 Non-linear fit with OLS 7
8.6 Kernel Ridge Regression 8
8.7 Example 11
8.8 Example: Discrete Radial Basis Function 11
8.9 Gaussian Process Regression: Mean and Variance 13

© Bjørk Hammer

October 7, 2026

Week 8

Regression

8.1 Ordinary Least Squares (OLS)

Minimization of Sum of Squared Residuals

Given N data points and p predictors, the linear regression model can be written in matrix form as:

$$\mathbf{y} = X\boldsymbol{\beta}$$

where:

- $\mathbf{y} \in \mathbb{R}^N$ is the vector of observed dependent variables (responses): $\mathbf{y} = [y_1, y_2, \ldots, y_N]^T$,
- $X \in \mathbb{R}^{N\times(p+1)}$ is the design matrix of independent variables (the first column is filled with ones for the intercept),
- $\boldsymbol{\beta} \in \mathbb{R}^{p+1}$ is the vector of regression coefficients: $\boldsymbol{\beta} = [\beta_0, \beta_1, \ldots, \beta_p]^T$.

The goal is to minimize the sum of squared residuals:

$$\min_{\boldsymbol{\beta}} \|\mathbf{y} - X\boldsymbol{\beta}\|^2$$

This objective function is the squared Euclidean norm of the residuals, and can be expanded as:

$$\begin{aligned} \|\mathbf{y} - X\boldsymbol{\beta}\|^2 &= (\mathbf{y} - X\boldsymbol{\beta})^T(\mathbf{y} - X\boldsymbol{\beta}) \\ &= \mathbf{y}^T\mathbf{y} - 2\mathbf{y}^T X\boldsymbol{\beta} + \boldsymbol{\beta}^T X^T X\boldsymbol{\beta} \end{aligned}$$

Next, take the derivative of the objective function with respect to β_i:

$$\frac{\partial}{\partial \beta_i}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 = -2\left[X^T\mathbf{y}\right]_i + 2\left[X^T X\boldsymbol{\beta}\right]_i$$

For $\boldsymbol{\beta}$ to be optimal, this must be equal to zero:

$$0 = -2\left[X^T\mathbf{y}\right]_i + 2\left[X^T X\boldsymbol{\beta}\right]_i$$

Meaning that:

$$\left[X^T X\boldsymbol{\beta}\right]_i = \left[X^T \mathbf{y}\right]_i$$

Since this applies for every entry in $\boldsymbol{\beta}$, i.e. for every value of i, one gets:

$$X^T X\boldsymbol{\beta} = X^T \mathbf{y}$$

Thus, the OLS solution is given by:

$$\boldsymbol{\beta} = (X^T X)^{-1} X^T \mathbf{y}$$

8.2 Maximization of Log-Likelihood

Gaussian Error Model

It is instructive to assume that the observed dependent variables contain errors, $\boldsymbol{\epsilon}$, that are normally distributed:

$$\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2 I)$$

where σ^2 is the variance of the error terms and I is the identity matrix.

The likelihood of observing the data $\mathbf{y}$ given the parameters $\boldsymbol{\beta}$ is:

$$p(\mathbf{y}|X, \boldsymbol{\beta}) = \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - X_i^T \boldsymbol{\beta})^2}{2\sigma^2}\right)$$

Taking the logarithm of the likelihood function:

$$\log \mathcal{L}(\boldsymbol{\beta}) = \sum_{i=1}^{N} \log\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - X_i^T \boldsymbol{\beta})^2}{2\sigma^2}\right)\right)$$

Simplifying the log-likelihood:

$$\log \mathcal{L}(\boldsymbol{\beta}) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{N} (y_i - X_i^T \boldsymbol{\beta})^2$$

Maximizing the log-likelihood is equivalent to minimizing the sum of squared residuals:

$$\min_{\boldsymbol{\beta}} \sum_{i=1}^{N} (y_i - X_i^T \boldsymbol{\beta})^2$$

This is exactly the same objective function as in OLS, which leads us back to the normal equation:

$$\boldsymbol{\beta} = (X^T X)^{-1} X^T \mathbf{y}$$

Hence, maximizing the log-likelihood of a Gaussian error model leads to the same solution as minimizing the sum of squared residuals.

8.3 Linear regression example

Example

Consider a simple linear regression with one predictor (i.e., $p = 1$). The design matrix X might look like this for three data points:

$$X = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}$$

The normal equation becomes:

$$\boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} = (X^T X)^{-1} X^T \mathbf{y}$$

Solving this system gives the best-fit intercept β_0 and slope β_1.

This matrix formulation of OLS is efficient for solving linear regression problems and forms the basis of more advanced methods in machine learning and statistical modeling.

8.4 Ridge Regression

Ridge Regression, also known as Tikhonov regularization, is a linear regression technique that includes a penalty term to prevent overfitting. The goal of Ridge Regression is to find a balance between fitting the data well and keeping the model coefficients small, which helps prevent large fluctuations in predictions due to small changes in the input.

Objective Function

Given a dataset with features $\mathbf{X} \in \mathbb{R}^{n \times p}$ and target values $\mathbf{y} \in \mathbb{R}^n$, the linear regression model predicts the target values using the formula:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta}$$

where $\boldsymbol{\beta} \in \mathbb{R}^p$ represents the model coefficients. The Ordinary Least Squares (OLS) solution minimizes the sum of squared residuals:

$$\min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$$

However, OLS can lead to overfitting, especially when there is multicollinearity or when the number of features p is large. To address this, Ridge Regression modifies the OLS objective function by adding an L_2 regularization term:

$$\min_{\boldsymbol{\beta}} \left(\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda \|\boldsymbol{\beta}\|^2 \right)$$

Here, $\lambda \geq 0$ is a regularization parameter that controls the trade-off between fitting the data (minimizing residuals) and keeping the model coefficients small (shrinking $\boldsymbol{\beta}$). When $\lambda = 0$, Ridge Regression reduces to OLS. As λ increases, the model coefficients shrink toward zero, reducing the variance of the model.

Closed-Form Solution

Ridge Regression has a closed-form solution, which can be derived by taking the gradient of the objective function and setting it to zero:

$$\boldsymbol{\beta}_{\text{ridge}} = \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}\right)^{-1} \mathbf{X}^\top \mathbf{y}$$

where $\mathbf{I}$ is the identity matrix of size $p \times p$. This solution is similar to OLS, but with an additional term $\lambda \mathbf{I}$ in the matrix inversion, which stabilizes the computation when $\mathbf{X}^\top \mathbf{X}$ is ill-conditioned or singular.

Impact of Regularization

The regularization term in Ridge Regression discourages large values of $\boldsymbol{\beta}$, helping to avoid overfitting. The parameter λ controls the strength of this penalty:

- If $\lambda = 0$, Ridge Regression becomes OLS, and the model may overfit the data.
- If λ is very large, the coefficients $\boldsymbol{\beta}$ shrink significantly, leading to underfitting.
- Typically, λ is chosen using cross-validation to find the best trade-off between bias and variance.

Example

Consider a simple regression problem with data points $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ and a linear model:

$$y = \beta_0 + \beta_1 x$$

Ridge Regression minimizes the following cost function:

$$\min_{\beta_0, \beta_1} \left(\sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2 + \lambda (\beta_0^2 + \beta_1^2) \right)$$

By solving this equation, we obtain the optimal values for β_0 and β_1, while regularization helps to prevent overfitting.

Conclusion

Ridge Regression introduces an L_2 penalty on the coefficients to address overfitting in linear models. This regularization technique is especially useful when multicollinearity is present or when the number of features is large compared to the number of observations. Ridge Regression serves as a foundation for more advanced techniques like Kernel Ridge Regression.

8.5 Non-linear fit with OLS

In ordinary least squares (OLS) regression, the design matrix can include non-linear terms to capture non-linear relationships between the independent variables and the dependent variable, even though the OLS method itself solves a linear equation in terms of the parameters. By introducing non-linear transformations of the original input variables as additional features, we can fit more complex, non-linear models.

Example: Quadratic Fit with Non-Linear Terms in OLS

Suppose we have one independent variable x, and we suspect that the relationship between x and the dependent variable y is quadratic. To fit a quadratic model, we can construct a design matrix that includes both linear and quadratic terms, such as:

$$\Phi(x) = \begin{pmatrix} 1 & \sqrt{2}x_1 & x_1^2 \\ 1 & \sqrt{2}x_2 & x_2^2 \\ 1 & \sqrt{2}x_3 & x_3^2 \\ \vdots & \vdots & \vdots \\ 1 & \sqrt{2}x_n & x_n^2 \end{pmatrix}$$

In this case:

- The first column corresponds to the intercept term (constant 1).
- The second column corresponds to the linear term, scaled by a factor $\sqrt{2}$ to standardize or balance the contributions of different features.
- The third column corresponds to the quadratic term, x^2, capturing the non-linear relationship.

We can write the model as:

$$
\begin{aligned}
y &= \beta_0 + \beta_1 \cdot \sqrt{2}x + \beta_2 \cdot x^2 \\
&= \begin{bmatrix} 1 & \sqrt{2}x & x^2 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}
\end{aligned}
$$

where β_0, β_1, and β_2 are the parameters to be estimated.

Solving the OLS Problem

Once the design matrix $\Phi(x)$ is constructed, the OLS problem is formulated as minimizing the sum of squared residuals:

$$\boldsymbol{\beta} = (\Phi^T \Phi)^{-1} \Phi^T y$$

where $\boldsymbol{\beta}$ is the vector of estimated parameters β_0, β_1, and β_2.

This allows us to fit a quadratic model to the data, even though the OLS method itself remains linear in the parameters. By introducing higher-order polynomial terms, we can extend this approach to fit any degree of polynomial, capturing increasingly complex relationships between x and y.

8.6 Kernel Ridge Regression

Non-linearity from data

Consider the previous example, where we had a model:

$$y_* = \begin{bmatrix} 1 & \sqrt{2}x_* & x_*^2 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

In that example, we assumed that the dependent variable had a constant, a linear, and a quadratic dependence on the independent variable. It would, however, be desirable if we did not have to make such a specific choice. To see how this can be avoided, it is instructive to first rewrite the model in the following way:

$$y_* = \phi(x_*) \cdot \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \beta_0 + \phi(x_*) \cdot \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \beta_1 + \phi(x_*) \cdot \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \beta_2$$

where ϕ is a mapping of the independent variable into a higher-dimensional space:

$$\phi(x) = \begin{bmatrix} 1 & \sqrt{2}x & x^2 \end{bmatrix}.$$

When written in this way, it becomes apparent that the fit picks up contributions of strength β_i in the various basis directions, e.g. $\begin{bmatrix}1 & 0 & 0\end{bmatrix}^T$, weighted by the dot-products between the transformed query point, $\phi(x_*)$, and those basis directions.

Now, if we abandon the normalized basis directions and instead use the directions of the transformed data points, a somewhat similar fit can be written:

$$y_* = \phi(x_*) \cdot \phi(x_1)\alpha_1 + \phi(x_*) \cdot \phi(x_2)\alpha_2 + \phi(x_*) \cdot \phi(x_3)\alpha_3 + \cdots + \phi(x_*) \cdot \phi(x_N)\alpha_N$$

where N is the number of data points. So far, we would still have to specify the mapping, ϕ, but we will eliminate this when **kernels** are introduced.

What has been obtained so far is:

- The mapping to the higher-dimensional space, ϕ, now only appears in dot-products.
- There are now as many coefficients as data points.

Kernel

The dot-products in the transformed high-dimensional space are called kernel values and are written as:

$$K(x_1, x_2) = \phi(x_1) \cdot \phi(x_2).$$

With this, the prediction is written:

$$y_* = \sum_i K(x_*, x_i)\alpha_i = K(x_*, \mathbf{X}^T)\boldsymbol{\alpha}.$$

Making the prediction for all the data points, we have:

$$y \simeq \mathbf{K}\boldsymbol{\alpha}$$

where $K_{ij} = K(x_i, x_j)$, and i, j run over the indices of all data points.

Finding the coefficients

A loss function can now be written:

$$\|\mathbf{y} - \mathbf{K}\boldsymbol{\alpha}\|^2 + \lambda\boldsymbol{\alpha}^T\mathbf{K}\boldsymbol{\alpha}$$

where the first term measures the deviation of the model from the values at the data points, and the second term penalizes large coefficients in $\boldsymbol{\alpha}$. This second term is not just $\|\boldsymbol{\alpha}\|^2$, reflecting that for directions that are shared by several data points, the coefficients must be suppressed further.

To find the coefficients, $\boldsymbol{\alpha}$, the first term of the loss function is rewritten:

$$\begin{aligned}\|\mathbf{y} - \mathbf{K}\boldsymbol{\alpha}\|^2 &= (\mathbf{y} - \mathbf{K}\boldsymbol{\alpha})^T(\mathbf{y} - \mathbf{K}\boldsymbol{\alpha}) \\ &= \mathbf{y}^T\mathbf{y} - \mathbf{y}^T\mathbf{K}\boldsymbol{\alpha} - \boldsymbol{\alpha}^T\mathbf{K}^T\mathbf{y} + \boldsymbol{\alpha}^T\mathbf{K}^T\mathbf{K}\boldsymbol{\alpha}\end{aligned}$$

The loss function can now be optimized by taking the derivative with respect to each of the entries in α_i:

$$\frac{d}{d\alpha_i}Loss = 0 - [\mathbf{y}^T\mathbf{K}]_i - [\mathbf{K}^T\mathbf{y}]_i + [\mathbf{K}^T\mathbf{K}\boldsymbol{\alpha}]_i + [\boldsymbol{\alpha}^T\mathbf{K}^T\mathbf{K}]_i + \lambda[\mathbf{K}\boldsymbol{\alpha}]_i + \lambda[\boldsymbol{\alpha}^T\mathbf{K}]_i$$

which when set equal to zero provides the solution:

$$\begin{aligned} 0 &= \frac{d}{d\alpha_i}Loss \\ \Rightarrow [\mathbf{K}^T\mathbf{y}]_i &= [\mathbf{K}^T\mathbf{K}\boldsymbol{\alpha}]_i + \lambda[\mathbf{K}\boldsymbol{\alpha}]_i \\ \Rightarrow [\mathbf{K}\mathbf{y}]_i &= [\mathbf{K}(\mathbf{K} + \lambda\mathbf{I})\boldsymbol{\alpha}]_i \end{aligned}$$

where the last line emerges from realizing that $\mathbf{K}^T = \mathbf{K}$. Since this result applies for all i, we have:

$$\mathbf{K}\mathbf{y} = \mathbf{K}(\mathbf{K} + \lambda\mathbf{I})\boldsymbol{\alpha}$$

which is fulfilled if either $\mathbf{K} = 0$ (which is not possible as its diagonal is always $\neq 0$) or:

$$\mathbf{y} = (\mathbf{K} + \lambda\mathbf{I})\boldsymbol{\alpha}$$

Finally, we have the solutions for the coefficients:

$$\boldsymbol{\alpha} = (\mathbf{K} + \lambda\mathbf{I})^{-1}\mathbf{y}$$

Recap

We now have the elements of the Kernel Ridge Regression method:

$$\begin{aligned} \boldsymbol{\alpha} &= (\mathbf{K} + \lambda\mathbf{I})^{-1}\mathbf{y} \\ y_* &= K(x_*, \mathbf{X}^T)\boldsymbol{\alpha} \end{aligned} \tag{8.1}$$

Remember that the kernel was introduced as a dot-product in the high-dimensional space that the independent variable(s), x, was projected to:

$$K_{ij} = \phi(x_i) \cdot \phi(x_j)$$

But observe, that the Kernel Ridge Regression solution, Eq. (8.1), does not refer explicitly to the mapping function, ϕ. This means that if the kernel is evaluated by some other means than a mapping to a high-dimensional space and the evaluation of the dot-product in that space, the non-linear fitting that was the goal can be obtained implicitly. This is the:

Kernel Trick

8.7 Example

Assume we are in a 1-dimensional space, i.e. we have one independent variable:

$$\mathbf{x} = \begin{bmatrix} x \end{bmatrix}$$

Then if the kernel is evaluated according to the square-kernel expression:

$$K_{ij} = \left(1 + \mathbf{x_i}^T \mathbf{x_j}\right)^2$$

we specifically get:

$$\begin{aligned} K(x_1, x_2) &= (1 + x_1 x_2)^2 \\ &= 1 + 2x_1 x_2 + x_1^2 x_2^2 \end{aligned}$$

which "happens to be" exactly the dot-product in the higher-dimensional space that ϕ maps x to:

$$\begin{aligned} \phi(x_1) \cdot \phi(x_2) &= \begin{bmatrix} 1 & \sqrt{2}x_1 & x_1^2 \end{bmatrix} \begin{bmatrix} 1 \\ \sqrt{2}x_2 \\ x_2^2 \end{bmatrix} \\ &= 1 + 2x_1 x_2 + x_1^2 x_2^2 \end{aligned}$$

Notice how the explicit projection and dot-product evaluation involves more computational evaluations that the implicit variant, using the square-kernel. This rationalizes why the kernel trick is useful.

If for some reason one would like to use kernel ridge regression without the implicit mapping to a higher-dimensional space, one may use the linear kernel:

$$K_{ij} = \mathbf{x_i}^T \mathbf{x_j}$$

8.8 Example: Discrete Radial Basis Function

By discretizing the x-axis and simplifying the radial basis functions, one can provide an illustration of the kernel trick for the Gaussian (RBF) kernel in a finite-dimensional space. This approach can give a concrete understanding of how the kernel trick avoids the need to explicitly calculate the feature mapping into the higher-dimensional space.

Step-by-Step Illustration of the Kernel Trick

- Discretizing the Input Space:

  Let's discretize the input space x such that:

  $$x = x_0 + i \cdot \Delta x,$$

  where i is an integer. For simplicity, assume that Δx is a fixed interval, and the input points lie on a discrete grid along the x-axis.

- Radial Basis Function (RBF) Approximation:

  The Gaussian kernel can be viewed as applying a radial basis function (RBF) to the input points. Normally, the RBF has a continuous shape, but for the sake of illustrating the point, here is a simplified version:

  $$\phi(x) = [\ldots, 0, 0, 0.1, 0.8, 0.1, 0, 0, \ldots]$$

  This simplified radial basis function has a peak value at the grid point x_i, which is closest to the input x, and smaller values at neighboring points. The function is centered on the closest grid point to x_1 and decays symmetrically around it.

- Mapping to a Higher-Dimensional Space:

  In this scenario, the feature space is the set of weights applied to these discretized radial basis functions. Suppose we have two points x_1 and x_2. Instead of working with the original 1D inputs, we map them into a higher-dimensional space where each dimension corresponds to the weight of the RBF at a particular grid point.

  For example, $\phi(x_1)$ might look like this in the higher-dimensional space:

  $$\phi(x_1) = [0, 0, 0.1, 0.8, 0.1, 0, 0, \ldots]$$

  And $\phi(x_2)$ might look like:

  $$\phi(x_2) = [0, 0.1, 0.8, 0.1, 0, 0, \ldots]$$

  where x_2 is centered around a different grid point.

- Calculating the Inner Product in the Higher-Dimensional Space:

  The dot product $\phi(x_1) \cdot \phi(x_2)$ would involve calculating the sum of element-wise products of these two vectors. However, since the radial basis functions are localized and decay quickly, many of the terms in this sum will be zero or very small. In this case, we're still computing an inner product in the transformed feature space.

- Using the Kernel Trick:

  Instead of explicitly computing the transformation into the higher-dimensional space and performing the dot product, the kernel trick allows us to compute the same result directly using the Gaussian kernel function:

  $$K(x_1, x_2) = \exp(-\frac{(x_1 - x_2)^2}{2\sigma^2}).$$

  The kernel function gives us the value of the inner product in the infinite-dimensional space without having to construct the explicit mapping. In our discretized example, this means that we don't need to store or compute the radial basis function values for each point; we can just evaluate the kernel function.

Key Insight

The kernel trick saves computation by directly providing the inner product between two points in the higher-dimensional space without needing to calculate their individual projections. In the Gaussian kernel case, the mapping would typically involve an infinite number of dimensions (basis functions), but the kernel function provides the result in constant time.

By discretizing the space and simplifying the RBFs, we've approximated the transformation into a finite-dimensional space. In this approximation, the kernel trick still avoids the need to explicitly construct and manipulate the feature vectors, illustrating its computational efficiency.

8.9 Gaussian Process Regression: Mean and Variance

In Gaussian Process Regression (GPR), the goal is to predict the value of a function at new points based on observed data. The predictions are in the form of a **mean** function and a **variance** function, which represent the most probable value of the function and the uncertainty at each prediction point, respectively.

Given Data

Assume we have observed data points $(X, \mathbf{y})$, where:

- $X = [x_1, x_2, \ldots, x_n]^\top$ are the input data points (of dimension $n \times d$, where d is the number of features),
- $\mathbf{y} = [y_1, y_2, \ldots, y_n]^\top$ are the corresponding target values.

We are interested in making predictions at new input points $X_* = [x_1^*, x_2^*, \ldots, x_m^*]^\top$.

Joint Distribution

We assume that the joint distribution of the observed outputs $\mathbf{y}$ and the predicted values $\mathbf{y}_*$ follows a multivariate Gaussian distribution:

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{y}_* \end{bmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K(X,X) + \sigma^2 I & K(X,X_*) \\ K(X_*,X) & K(X_*,X_*) \end{bmatrix}\right)$$

where:

- $K(X, X)$ is the covariance matrix of the training data, given by the kernel function $K(x, x')$ applied to each pair of points in X.
- $K(X, X_*)$ is the covariance between the training points and the new points.
- $K(X_*, X_*)$ is the covariance matrix of the new points.
- σ^2 is the noise variance in the observed data.

Mean and Variance of Predictions

The conditional distribution of the predicted values $\mathbf{y}_*$ given the observed data $(X, \mathbf{y})$ is Gaussian with the following **mean** and **variance**:

Mean:

$$\mathbb{E}[\mathbf{y}_*] = \mathbf{m}_* = K(X_*, X) \left(K(X, X) + \sigma^2 I \right)^{-1} \mathbf{y}$$

This gives the predicted mean (or expected value) at the new points X_*.

Variance:

$$\text{Var}[\mathbf{y}_*] = K(X_*, X_*) - K(X_*, X) \left(K(X, X) + \sigma^2 I \right)^{-1} K(X, X_*)$$

This provides the uncertainty (variance) of the predictions at the new points X_*.

Interpretation

- The predicted **mean** function $\mathbf{m}_*$ is a linear combination of the kernel function applied to the training points and the new points.
- The predicted **variance** gives a measure of how uncertain we are about the predicted values, which depends on the distance between the new points and the observed data, as captured by the kernel function.

In practice, Gaussian Process Regression provides both the prediction and the uncertainty around the prediction, which makes it a powerful tool for regression tasks where uncertainty estimation is important.